Report 2: UML Class Diagram Report

Hospital Management System (HMS)

Institution	Kigali Independent University (ULK)
Programme	Master of Software Engineering (MSSE)
Assignment	UML Class Diagram Modelling
Instructor	Eric Maniraguha
Group members	Aser (Student ID: 202652088) [Partner full name] (Student ID: [Partner student ID])
Submission deadline	Tuesday, 20 October 2026, 8:30 AM

1. Introduction

This report presents the UML class diagram of the Hospital Management System (HMS). It lists the 13 classes with their attributes and methods, the 15 relationships between them with multiplicities, and a short explanation of each relationship. The business context, scope and requirements are given in Report 1.

Notation used

- A class is a box with three compartments: name, attributes and methods. **-** means private and **+** means public.
- **Generalization** (hollow triangle at the parent): an "is-a" relationship with inheritance.
- **Aggregation** (hollow diamond at the whole): a "has-a" relationship where the parts can exist on their own.
- **Composition** (filled diamond at the whole): a "has-a" relationship where the parts cannot exist without the whole.
- **Association** (plain line): classes that work together; the verb on the line names the relationship.
- **Multiplicity**: 1 = exactly one, 0..1 = zero or one, 0..* = zero or many, 1..* = one or many.

2. UML Class Diagram

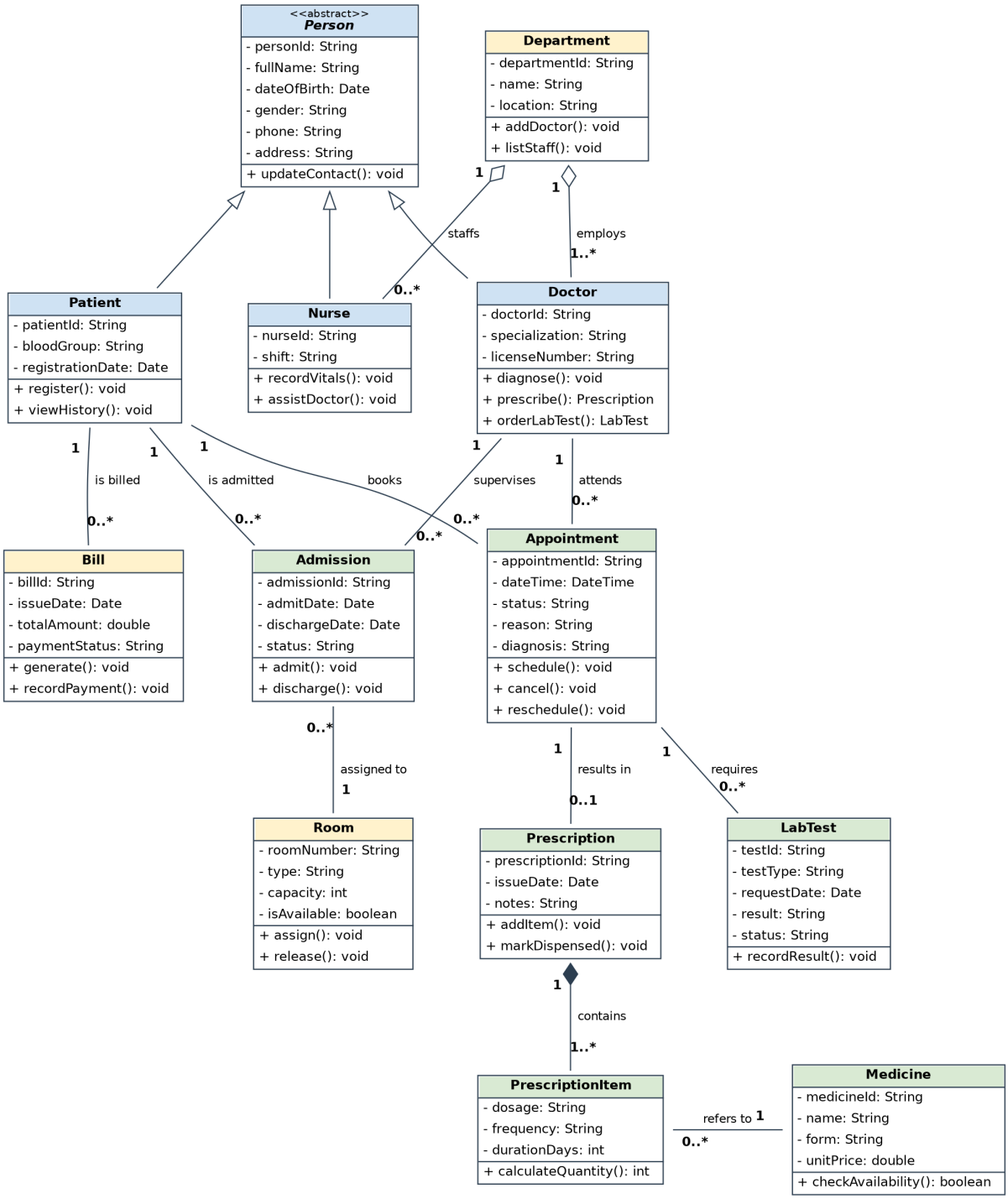


Figure 1: UML class diagram of the Hospital Management System.

3. Classes, Attributes and Methods

Class	Attributes	Methods
Person *(abstract)*	- personId: String - fullName: String - dateOfBirth: Date - gender: String - phone: String - address: String	+ updateContact(): void
Patient	- patientId: String - bloodGroup: String - registrationDate: Date	+ register(): void + viewHistory(): void
Doctor	- doctorId: String - specialization: String - licenseNumber: String	+ diagnose(): void + prescribe(): Prescription + orderLabTest(): LabTest
Nurse	- nurseId: String - shift: String	+ recordVitals(): void + assistDoctor(): void
Department	- departmentId: String - name: String - location: String	+ addDoctor(): void + listStaff(): void
Appointment	- appointmentId: String - dateTime: DateTime - status: String - reason: String - diagnosis: String	+ schedule(): void + cancel(): void + reschedule(): void
Prescription	- prescriptionId: String - issueDate: Date - notes: String	+ addItem(): void + markDispensed(): void
PrescriptionItem	- dosage: String - frequency: String - durationDays: int	+ calculateQuantity(): int
Medicine	- medicineId: String - name: String - form: String - unitPrice: double	+ checkAvailability(): boolean
LabTest	- testId: String - testType: String - requestDate: Date - result: String - status: String	+ recordResult(): void
Admission	- admissionId: String - admitDate: Date - dischargeDate: Date - status: String	+ admit(): void + discharge(): void
Room	- roomNumber: String - type: String - capacity: int - isAvailable: boolean	+ assign(): void + release(): void
Bill	- billId: String - issueDate: Date - totalAmount: double - paymentStatus: String	+ generate(): void + recordPayment(): void

4. Relationships and Multiplicities

#	Relationship	Type	Multiplicity
1	Patient is-a Person	Generalization	-
2	Doctor is-a Person	Generalization	-
3	Nurse is-a Person	Generalization	-
4	Department - Doctor (employs)	Aggregation	Department **1** : **1..*** Doctor
5	Department - Nurse (staffs)	Aggregation	Department **1** : **0..*** Nurse
6	Patient - Appointment (books)	Association	Patient **1** : **0..*** Appointment
7	Doctor - Appointment (attends)	Association	Doctor **1** : **0..*** Appointment
8	Appointment - LabTest (requires)	Association	Appointment **1** : **0..*** LabTest
9	Appointment - Prescription (results in)	Association	Appointment **1** : **0..1** Prescription
10	Prescription - PrescriptionItem (contains)	Composition	Prescription **1** : **1..*** PrescriptionItem
11	PrescriptionItem - Medicine (refers to)	Association	PrescriptionItem **0..*** : **1** Medicine
12	Patient - Admission (is admitted)	Association	Patient **1** : **0..*** Admission
13	Admission - Room (assigned to)	Association	Admission **0..*** : **1** Room
14	Doctor - Admission (supervises)	Association	Doctor **1** : **0..*** Admission
15	Patient - Bill (is billed)	Association	Patient **1** : **0..*** Bill

5. Explanation of the Relationships

5.1 Generalization (inheritance)

Person is an abstract class holding the attributes every human actor shares. Patient, Doctor and Nurse inherit them, which avoids repeating personId, fullName, dateOfBirth, gender, phone and address in three classes. Person cannot be instantiated on its own; every real person in the system is a patient, a doctor or a nurse.

5.2 Aggregation

- **Department - Doctor:** A department employs one or more doctors; each doctor belongs to exactly one department. Doctors continue to exist if a department is reorganised, so this is aggregation.
- **Department - Nurse:** A department has zero or more nurses assigned to it; each nurse works in exactly one department.

5.3 Composition

- **Prescription - PrescriptionItem:** A prescription contains one or more items; an item cannot exist without its prescription, so deleting the prescription deletes its items (composition).

5.4 Associations

- **Patient - Appointment (books):** A patient can book zero or many appointments; each appointment is booked by exactly one patient.
- **Doctor - Appointment (attends):** A doctor attends zero or many appointments; each appointment is attended by exactly one doctor.
- **Appointment - LabTest (requires):** An appointment may require zero or more lab tests; each lab test is requested during exactly one appointment.

- **Appointment - Prescription (results in):** An appointment results in at most one prescription; each prescription belongs to exactly one appointment.
- **PrescriptionItem - Medicine (refers to):** Each prescription item refers to exactly one medicine, while a medicine may appear in many items. Together with the composition above, PrescriptionItem resolves the many-to-many relationship between Prescription and Medicine.
- **Patient - Admission (is admitted):** A patient may be admitted zero or more times; each admission concerns exactly one patient.
- **Admission - Room (assigned to):** Each admission is assigned to exactly one room, while a room hosts many admissions over time.
- **Doctor - Admission (supervises):** A doctor supervises zero or more admissions; each admission is supervised by exactly one doctor.
- **Patient - Bill (is billed):** A patient may receive zero or more bills; each bill is addressed to exactly one patient.

5.5 Resolving a many-to-many relationship

A prescription can contain many medicines, and a medicine can appear on many prescriptions. A direct many-to-many link cannot store the dosage, frequency or duration of each medicine. The class **PrescriptionItem** therefore sits between Prescription and Medicine: it carries these details, is owned by exactly one prescription (composition) and refers to exactly one medicine (association). This is the same idea as the Enrollment class between Student and Course.

6. Design Decisions

- **Diagnosis stored in Appointment.** A separate Consultation class was avoided to keep the model readable; it could be added later.
- **Billing at patient level.** A Bill is addressed to a patient; its total is computed from appointments, lab tests, medicines and admissions through the generate() method.
- **Aggregation for Department.** Doctors and nurses outlive a reorganisation of departments, so they are not owned by the department.
- **Composition for PrescriptionItem.** Items have no meaning without their prescription.
- **Methods are business operations** (for example schedule(), discharge(), recordPayment()) rather than simple getters and setters.

7. Conclusion

The class diagram models the main entities of a hospital and the way they interact: people, appointments and treatments, admissions and rooms, and billing. It uses generalization, aggregation, composition and association with explicit multiplicities, and it can be refined into a database schema or an object-oriented implementation.